3D Population Density Mapping of Nepal

Using Rayshader for Advanced Geospatial Visualization

Pukar Bhandari

pukar.bhandari@outlook.com

2023-01-03

Table of contents

1 Project Overview 1
2 Environment Setup 2
2.1 Package Installation and Loading 2
3 Data Acquisition and Loading 3
3.1 Population Hexagon Data 3
4 Data Processing and Visualization Setup 4
4.1 Initial Data Exploration 4
4.2 Raster Conversion and Matrix Preparation 5
5 Color Scheme Development 6
5.1 MetBrewer Color Palette Selection 6
6 3D Visualization Rendering 7
6.1 Interactive 3D Display 7
6.2 High-Quality Rendering 8
7 Post-Processing and Annotation 8
7.1 Geographic Labeling 8
8 Technical Methodology 12
8.1 Spatial Data Processing Pipeline 12
8.2 Rendering Specifications 12
9 Results and Applications 13
10 Conclusion 13

1 Project Overview

This analysis demonstrates advanced geospatial visualization techniques using R's rayshader package to create a stunning 3D population density map of Nepal. The project showcases the integration of high-resolution population data with topographic visualization methods, resulting in an interactive and informative representation of demographic patterns across Nepal's diverse terrain.

Key Technologies: R, rayshader, sf, stars, ragg, grid
Data Source: Kontur Population Dataset (400m H3 hexagon resolution)
Projection: ESRI:102306 (Nepal Nagarkot TM)

2 Environment Setup

2.1 Package Installation and Loading

```
# Core packages for geospatial analysis
install.packages(c("here", "sf", "ggplot2", "stars"))

# Visualization packages
install.packages(c("rayshader", "MetBrewer", "colorspace", "rayrender"))

# Annotation: PNG reading, anti-aliased PNG device, and font registration
install.packages(c("png", "ragg", "systemfonts"))
```

```
# Load required libraries
library(here)
library(sf)
library(stars)
library(rayshader)
library(MetBrewer)
library(colorspace)
library(rayrender)

# Annotation: PNG reading, anti-aliased PNG device, and font registration
# (grid ships with base R)
library(png)
library(ragg)
library(systemfonts)
library(grid)
```

```
# Configure 3D rendering
options(rgl.useNULL = FALSE)

project_crs <- "EPSG:6207"
```

3 Data Acquisition and Loading

3.1 Population Hexagon Data

The analysis utilizes high-resolution population data from Kontur, provided in H3 hexagonal grid format at 400-meter resolution. This dataset offers superior granularity compared to traditional administrative boundary-based population estimates.

```
# Load population data (H3 hexagons at 400m resolution)
nep_hex <- sf::st_read(
  paste0(
    "/vsigzip/",
                                here::here("posts/nepal-population-density/data/
kontur_population_NP_20220630.gpkg.gz")
  )
) |>
  sf::st_transform(project_crs)
```

```
Reading layer `population' from data source
    `/vsigzip/D:/GitHub/ar-puuk.github.io/posts/nepal-population-density/data/
kontur_population_NP_20220630.gpkg.gz'
  using driver `GPKG'
Simple feature collection with 122809 features and 2 fields
Geometry type: POLYGON
Dimension:     XY
Bounding box:  xmin: 8911286 ymin: 3041698 xmax: 9818046 ymax: 3562066
Projected CRS: WGS 84 / Pseudo-Mercator
```

```
# Load administrative boundaries
nep_admin <- sf::st_read(
  paste0(
    "/vsigzip/",
                                here::here("posts/nepal-population-density/data/
kontur_boundaries_NP_20220407.gpkg.gz")
  )
) |>
  sf::st_transform(project_crs)
```

```
Reading layer `sql_statement' from data source
    `/vsigzip/D:/GitHub/ar-puuk.github.io/posts/nepal-population-density/data/
kontur_boundaries_NP_20220407.gpkg.gz'
  using driver `GPKG'
Simple feature collection with 4902 features and 5 fields
```

```
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: 80.05862 ymin: 26.34776 xmax: 88.20153 ymax: 30.44694
Geodetic CRS:  WGS 84
```

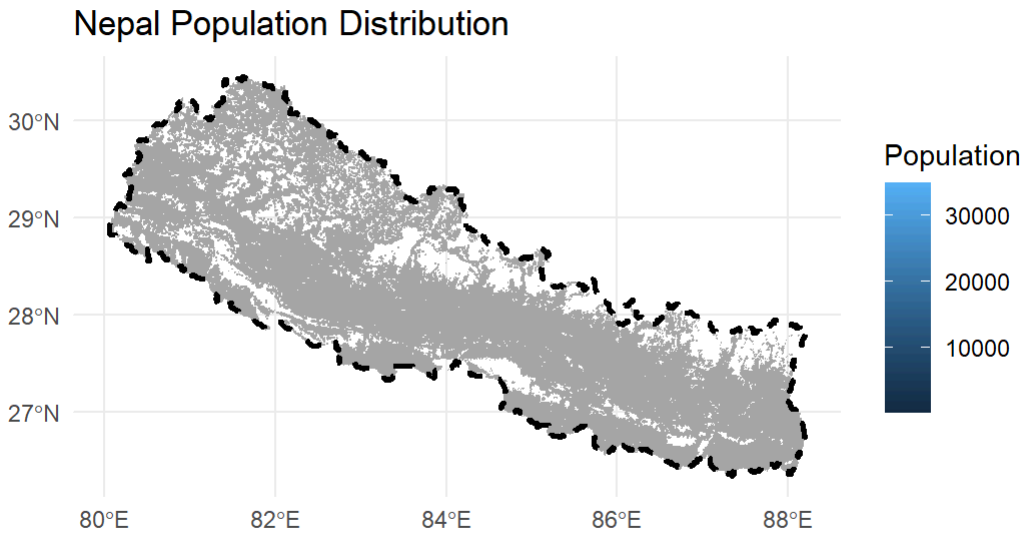
```
# Create unified Nepal boundary
nepal_boundary <- nep_admin |>
  sf::st_geometry() |>
  sf::st_union() |>
  sf::st_sf() |>
  sf::st_make_valid()
```

> **Note**
>
> **Data Sources**:
>
> - **Population Data**: Kontur Population Dataset
> - **Administrative Boundaries**: Kontur Boundaries Dataset
> - **Coordinate Reference System**: EPSG:6207 for accurate representation of Nepal's geography

4 Data Processing and Visualization Setup

4.1 Initial Data Exploration

```
# Preliminary visualization to verify data quality
ggplot2::ggplot(nep_hex) +
  ggplot2::geom_sf(ggplot2::aes(fill = population), color = "gray66", linewidth
= 0) +
  ggplot2::geom_sf(data = nepal_boundary, fill = NA, color = "black",
          linetype = "dashed", linewidth = 1) +
  ggplot2::theme_minimal() +
  ggplot2::labs(title = "Nepal Population Distribution",
       fill = "Population")
```



4.2 Raster Conversion and Matrix Preparation

The conversion from vector hexagon data to raster format is essential for rayshader's 3D rendering capabilities. This process involves calculating optimal dimensions while preserving the geographic aspect ratio.

```
# Calculate bounding box and aspect ratio
bbox <- sf::st_bbox(nepal_boundary)

# Define corner points for aspect ratio calculation
bottom_left <- sf::st_point(c(bbox[["xmin"]], bbox[["ymin"]])) |>
  sf::st_sfc(crs = project_crs)
bottom_right <- sf::st_point(c(bbox[["xmax"]], bbox[["ymin"]])) |>
  sf::st_sfc(crs = project_crs)
top_left <- sf::st_point(c(bbox[["xmin"]], bbox[["ymax"]])) |>
  sf::st_sfc(crs = project_crs)

# Calculate dimensions
width <- sf::st_distance(bottom_left, bottom_right)
height <- sf::st_distance(bottom_left, top_left)

# Determine aspect ratios
if(width > height) {
  w_ratio <- 1
  h_ratio <- height / width
```

```
} else {
  h_ratio <- 1
  w_ratio <- width / height
}

# Convert to raster with optimal resolution
size <- 1000 * 2.5

pop_raster <- stars::st_rasterize(nep_hex,
                          nx = floor(size * w_ratio),
                          ny = floor(size * h_ratio))

# Create matrix for 3D rendering
pop_matrix <- matrix(pop_raster$population,
                     nrow = floor(size * w_ratio),
                     ncol = floor(size * h_ratio))
```

5 Color Scheme Development

5.1 MetBrewer Color Palette Selection

The visualization employs the "OKeeffe2" palette from MetBrewer, chosen for its warm earth tones that effectively represent population density while maintaining visual appeal and accessibility.

```
# Generate color palette
color <- MetBrewer::met.brewer("OKeeffe2")

# Create texture gradient with bias toward higher values
texture <- grDevices::colorRampPalette(color, bias = 3)(256)

# Preview color scheme
colorspace::swatchplot(texture)
```



6 3D Visualization Rendering

6.1 Interactive 3D Display

```
# Close any existing RGL windows
rgl::close3d()

# Generate 3D visualization
pop_matrix |>
  rayshader::height_shade(texture = texture) |>
  rayshader::plot_3d(
    heightmap = pop_matrix,
    zscale = 250 / 2.5,
    solid = FALSE,
    shadowdepth = 0
  )

# Set optimal camera position
rayshader::render_camera(
  theta = 0,
  phi = 30,
  zoom = 0.4,
  fov = 90
)
```

6.2 High-Quality Rendering

The final rendering process utilizes rayshader's advanced lighting system to create publication-ready visualizations with professional-grade quality and resolution.

```
outfile <- here::here("posts/nepal-population-density/plots/final_plot.png")

# Render high-quality image with timing
{
  start_time <- Sys.time()
  cat(crayon::cyan("Rendering started:", start_time), "\n")

  # Ensure output directory exists
  if(!file.exists(outfile)) {
    png::writePNG(matrix(1), target = outfile)
  }

  rayshader::render_highquality(
    filename = outfile,
    interactive = FALSE,
    lightdirection = 225,
    lightaltitude = c(20, 80),
    lightcolor = c(color[2], "white"),
    lightintensity = c(600, 100),
    width = 1980,
    height = 1080,
    samples = 300
  )

  end_time <- Sys.time()
  diff <- end_time - start_time
  cat(crayon::cyan("Rendering completed in:", diff), "\n")
}
```

```
Rendering started: 2026-11-12 23:17:57.292951
```

```
Rendering completed in: 10.8995585481326
```

7 Post-Processing and Annotation

7.1 Geographic Labeling

The final step adds the title, city labels and attribution on top of the rendered image. Instead of {magick}, it uses base {grid} for layout and the {ragg} PNG device for anti-aliased text. Two font

files in the `font/` folder are registered for the session with {systemfonts}, so nothing needs to be installed in Windows: Ananda Namaste for the title, and Jost Medium (a free Google Fonts typeface in the style of Futura) for everything else.

```
# Register the bundled font (no system-wide install required)
systemfonts::register_font(
  "Ananda Namaste",
    plain  =  here::here("posts/nepal-population-density/font/Ananda  Namaste
400.ttf")
)

# Jost (Google Fonts, OFL) stands in for Futura BT Medium, which is commercial
systemfonts::register_font(
  "Jost Medium",
  plain = here::here("posts/nepal-population-density/font/Jost-500.ttf")
)

# Define text styling
text_color <- colorspace::darken(color[7], .5)
colorspace::swatchplot(text_color)
```



```
# Load rendered image
render <- png::readPNG(
  here::here("posts/nepal-population-density/plots/final_plot.png")
```

```
)
img_h <- dim(render)[1]
img_w <- dim(render)[2]

# Place text using ImageMagick-style gravity and pixel offsets
# (offsets move the label inward from the named edge; for "center", +x is right
and +y is down)
add_label <- function(label, gravity, dx, dy, size, alpha = 0.8,
                      family = "Jost Medium") {
  pos <- switch(
    gravity,
    northeast = list(x = img_w - dx,      y = dy,              h = 1,   v = 1),
    east      = list(x = img_w - dx,      y = img_h / 2 + dy, h = 1,   v = 0.5),
    west      = list(x = dx,              y = img_h / 2 + dy, h = 0,   v = 0.5),
    center    = list(x = img_w / 2 + dx,  y = img_h / 2 + dy, h = 0.5, v = 0.5),
    southwest = list(x = dx,              y = img_h - dy,     h = 0,   v = 0)
  )
  grid::grid.text(
    label,
    x = grid::unit(pos$x, "native"),
    y = grid::unit(pos$y, "native"),
    hjust = pos$h,
    vjust = pos$v,
    gp = grid::gpar(
      col = grDevices::adjustcolor(text_color, alpha.f = alpha),
      fontsize = size,
      fontfamily = family
    )
  )
}
```

```
# res = 72 makes one point equal one pixel, so sizes match the image's pixel grid
ragg::agg_png(
  here::here("posts/nepal-population-density/nepal_population_density.png"),
  width = img_w, height = img_h, units = "px", res = 72
)

grid::grid.newpage()
grid::grid.raster(render, width = 1, height = 1)

# Native units with y running top-down, to match pixel offsets
grid::pushViewport(grid::viewport(xscale = c(0, img_w), yscale = c(img_h, 0)))
```

```r
# Title: scale "NEPAL" so its width matches the subtitle's width
label_width <- function(label, size, family) {
  grid::convertWidth(
    grid::grobWidth(grid::textGrob(
      label, gp = grid::gpar(fontsize = size, fontfamily = family)
    )),
    "native", valueOnly = TRUE
  )
}

subtitle_size <- 36.5
subtitle_width <- label_width("POPULATION DENSITY MAP", subtitle_size, "Jost Medium")
# Ananda Namaste has side bearings and an overhanging top bar on the "n", so the
# advance width is nudged up (x1.04) to line the stem of the "n" up with the "P"
title_size <- 1.04 * 100 * subtitle_width / label_width("NEPAL", 100, "Ananda Namaste")

add_label("NEPAL", "northeast", 50, 65, title_size, alpha = 1, family = "Ananda Namaste")
add_label("POPULATION DENSITY MAP", "northeast", 50, 235, subtitle_size, alpha = 1)

# Major urban centers
add_label("Kathmandu", "center", 250, -50, 30)
add_label("Pokhara", "center", -30, 35, 25)
add_label("Biratnagar", "east", 125, 100, 28)

# Regional centers
add_label("Birgunj", "center", 130, 100, 25)
add_label("Nepalgunj", "center", -450, 0, 24)
add_label("Janakpur", "east", 500, 140, 22)

# Additional urban areas
add_label("Itahari", "east", 230, 150, 24)
add_label("Surkhet", "center", -450, -75, 20)
add_label("Tikapur", "west", 400, -125, 22)
add_label("Dhangadhi", "west", 275, -135, 18)
add_label("Bharatpur", "center", 60, 0, 20)
add_label("Butwal", "center", -125, 5, 20)
add_label("Ghorahi
Tulsipur", "center", -350, -100, 22)
```

```
# Attribution
add_label(
   "Visualization by: Pukar Bhandari with Rayshader(@tylermorganwall) | Data:
Kontur Population (Released 2022-06-30)",
  "southwest", 20, 20, 20, alpha = 0.6
)

invisible(grid::popViewport())
invisible(dev.off())
```

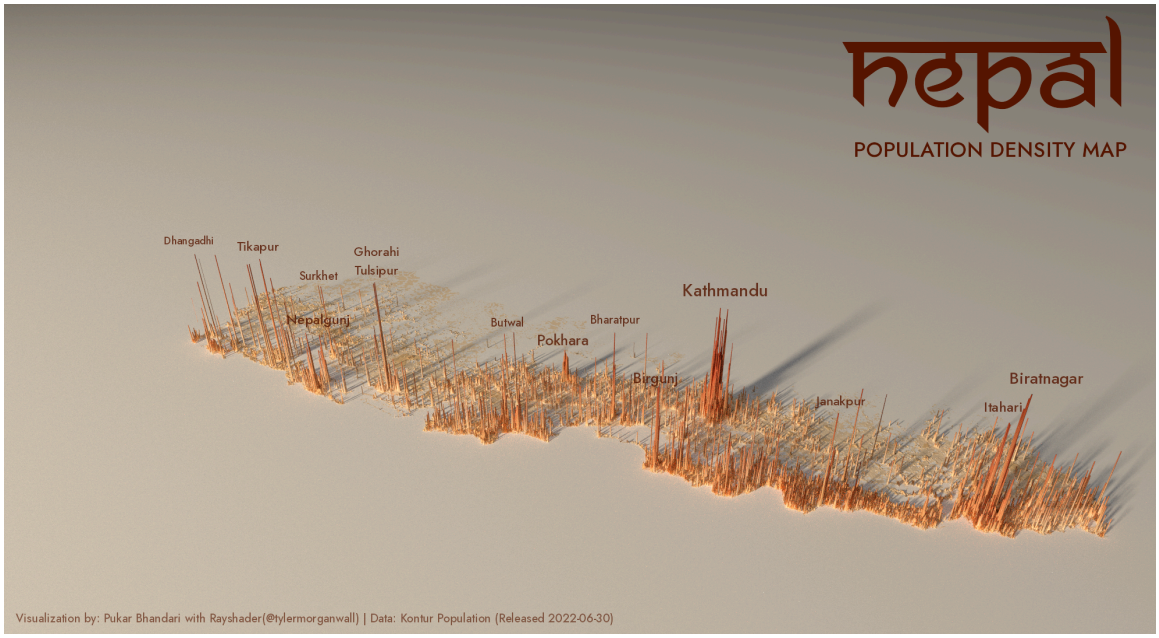


Figure 1: Population Density of Nepal

8 Technical Methodology

8.1 Spatial Data Processing Pipeline

1. **Data Transformation**: Vector hexagon data converted to standardized coordinate reference system (Nepal Nagarkot TM - ESRI:102306)
2. **Rasterization**: Optimal grid resolution calculated to balance detail with computational efficiency
3. **Matrix Conversion**: Raster data transformed into height matrix for 3D rendering
4. **Texture Mapping**: Population values mapped to color gradients using perceptually uniform color spaces

8.2 Rendering Specifications

- **Resolution**: 1980×1080 pixels for high-definition output

- **Sampling**: 300 samples for anti-aliasing and smooth gradients
- **Lighting**: Dual-source lighting system (directional + ambient)
- **Z-Scale**: Optimized vertical exaggeration (250/2.5) for clear population peaks

9 Results and Applications

This methodology produces publication-quality 3D population density visualizations suitable for:

- **Urban Planning**: Identifying population clusters and growth patterns
- **Infrastructure Development**: Targeting high-density areas for transportation networks
- **Emergency Response**: Understanding population distribution for resource allocation
- **Academic Research**: Visual communication of demographic data

10 Conclusion

This workflow demonstrates the power of combining modern R packages for geospatial analysis with advanced 3D visualization techniques. The resulting maps provide intuitive understanding of Nepal's population distribution patterns, highlighting the concentration of people in the Terai region and major urban centers while showcasing the relatively sparse population in mountainous areas.

The methodology is transferable to other geographic contexts and can be adapted for various demographic or infrastructure planning applications in transportation and urban development projects.

Want to contribute or suggest improvements? Visit the project repository at: https://github.com/arpuuk/Population-Density-Maps